



HOCHSCHULE
RAVENSBURG-WEINGARTEN
UNIVERSITY
OF APPLIED SCIENCES

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens

christoph.andriessens@hs-weingarten.de



┌ EINFÜHRUNG: VOM PROGRAMMIEREN ÜBER DAS SOFTWARESYSTEM ZU SOFTWARE ENGINEERING

Wie alles anfängt

- So fangen wir alle an: Wir lernen, in irgendeiner Programmiersprache ein paar Codezeilen aufschreiben
 - Ein kleines, hypothetisches, wohldefiniertes Problem
 - Code muss kaum gegliedert werden, er ist sehr übersichtlich
- Dann läuft's nicht beim ersten Mal ... und vielleicht auch nicht beim 5. Mal!
 - Wir probieren verschiedene Eingaben aus („testen“)
 - Wir denken wieder über das Problem nach
 - Wir erarbeiten uns Möglichkeiten, wie wir Fehler suchen und finden
 - Wir merken, dass wir nicht alles durchdacht haben
 - Und irgendwann beschließen wir: Jetzt ist's genug

Vielleicht in
anderer
Reihenfolge

Eine „einfache“ Reihenfolge von Schritten (1/4)

1. Das **Problem verstehen** – „Anforderungen“
 - **Funktionen**, die die Software bieten soll
 - Andere Dinge, die die Software auch können soll, die aber keine „Funktionen“ sind – **nicht-funktionales**: Performance, Speicherverbrauch, Sicherheit, etc.

Eine „einfache“ Reihenfolge von Schritten (2/4)

2. Etwas **entwerfen** – um das Problem zu lösen
 - Die Funktionen in eine **Reihenfolge** bringen, vielleicht zeichnen wir ein Diagramm
 - Wir überlegen: Was soll das Programm als **Eingabe** erhalten, was soll es **ausgeben**?
 - In welcher **Darstellung** und welchem **Format**?
 - Was müssen wir berücksichtigen bei Performance etc. (dem „**nicht-funktionalen**“), wie startet man das Programm, wie verwendet man es (**Benutzeroberfläche**), ...
 - Wollen wir vielleicht einen speziellen **Algorithmus** nutzen?

Eine „einfache“ Reihenfolge von Schritten (3/4)

3. Implementieren / codieren: Zum Entwurf den tatsächlichen Programmtext schreiben

- Je nach Entwurf und Programmiersprache direkte Umsetzung oder noch etwas mehr Entwurf
 - Spezifische Eingabe und Ausgabe mit konkreten Formaten
 - Die Verarbeitungsschritte in die richtige Reihenfolge bringen und mit den passenden Sprachkonstrukten umsetzen
 - Dabei herausfinden, wie man Bibliotheksfunktionen der Programmiersprache richtig benutzt

Eine „einfache“ Reihenfolge von Schritten (4/4)

4. Ausprobieren! (**Validieren** / **Testen**)

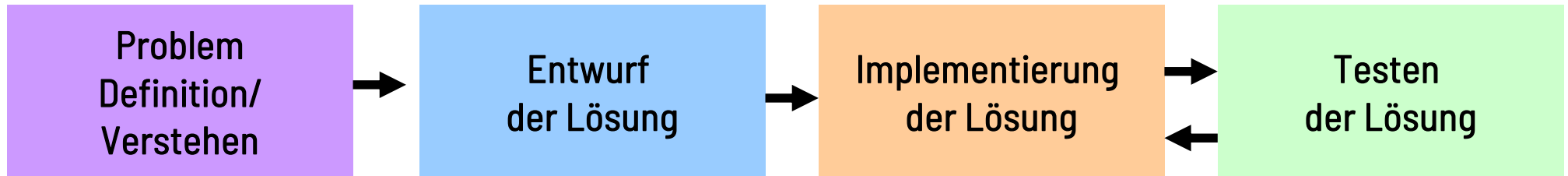
Das Programm starten, die Ausgaben prüfen und mit Erwartungen vergleichen

- Die Eingaben dazu sind „Testfälle“ – und erfordern manchmal nachdenken
- Falls die Ausgaben nicht die Erwartungen erfüllen, dann:
 - Fehler suchen
 - Fehler beheben
 - Wieder ausführen, ...
- Wir hören auf, wenn alle „Testfälle“ die erwarteten Resultate bringen

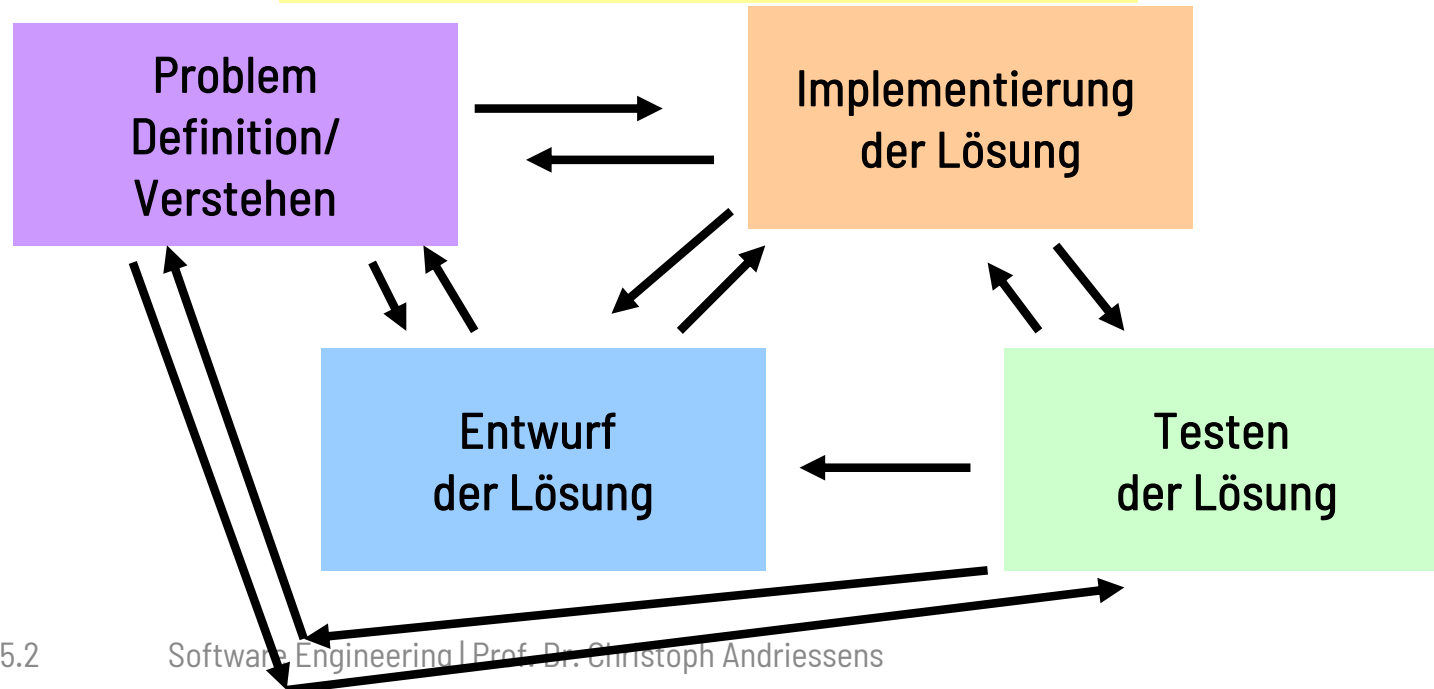
Und wieviele „Testfälle“ sollten wir uns ausdenken und ausprobieren?

Was passiert wirklich?

"Gedacht" – Ideale Situation



"Tatsächlich" – was wirklich geschieht



Programm fertig!

Worauf kommt es sonst noch an?

- **Wie lange** (verstrichene Zeit) hat die Fertigstellung gedauert?
- **Wieviel Aufwand** (etwa Summe an Personenstunden) wurde für die Arbeit aufgewendet?
- Löst das Ergebnis das **vollständige** Problem?
- **Wie „gut“** ist das Ergebnis – (Quelltext, Entwurf, Dokumentation, Test etc.)?

basierend worauf?

Beispiel-Problem

Aufgabe ist ein Programm in Ihrer favorisierten Programmiersprache, das Zahlen als Eingabe akzeptiert, den Durchschnitt ausrechnet und das Ergebnis ausgibt.

- **Stellen Sie sich folgende Fragen:**
 - Wie lange (in verstrichener Zeit) werden Sie brauchen, um die Lösung zu implementieren?
 - Wieviel tatsächlichen Aufwand (Personenstunden) werden Sie innerhalb dieses Zeitraums aufwenden?
 - Wie gut wird Ihre Lösung das Problem lösen (weshalb)?
 - Wie gut ist ihr Quelltext / Entwurf / Dokumentation / Test?

Ein ganzes „System“ bauen

Jetzt schreiben wir nicht nur ein Programm, sondern bauen ein „Software-System“.

- *Im Beispiel: Anstatt einer Funktion zur Berechnung des Durchschnitts erstellen wir jetzt eine Tabellenkalkulation mit mehrseitigen Berechnungstabellen und 80 teilweise sehr komplexen Berechnungsfunktionen.*

Was ist der Unterschied?

➤ Größe und KOMPLEXITÄT

➤ in der **Breite**

➤ in der **Tiefe**

Komplexitätszunahme in der Breite

- Mehr **Funktionen**
- Mehr **Schnittstellen**, intern und zu anderen Systemen
- Mehr **Benutzer**, gleichzeitige und unterschiedlichere Benutzer
- Mehr **Daten**, unterschiedlichere Daten und Datenstrukturen

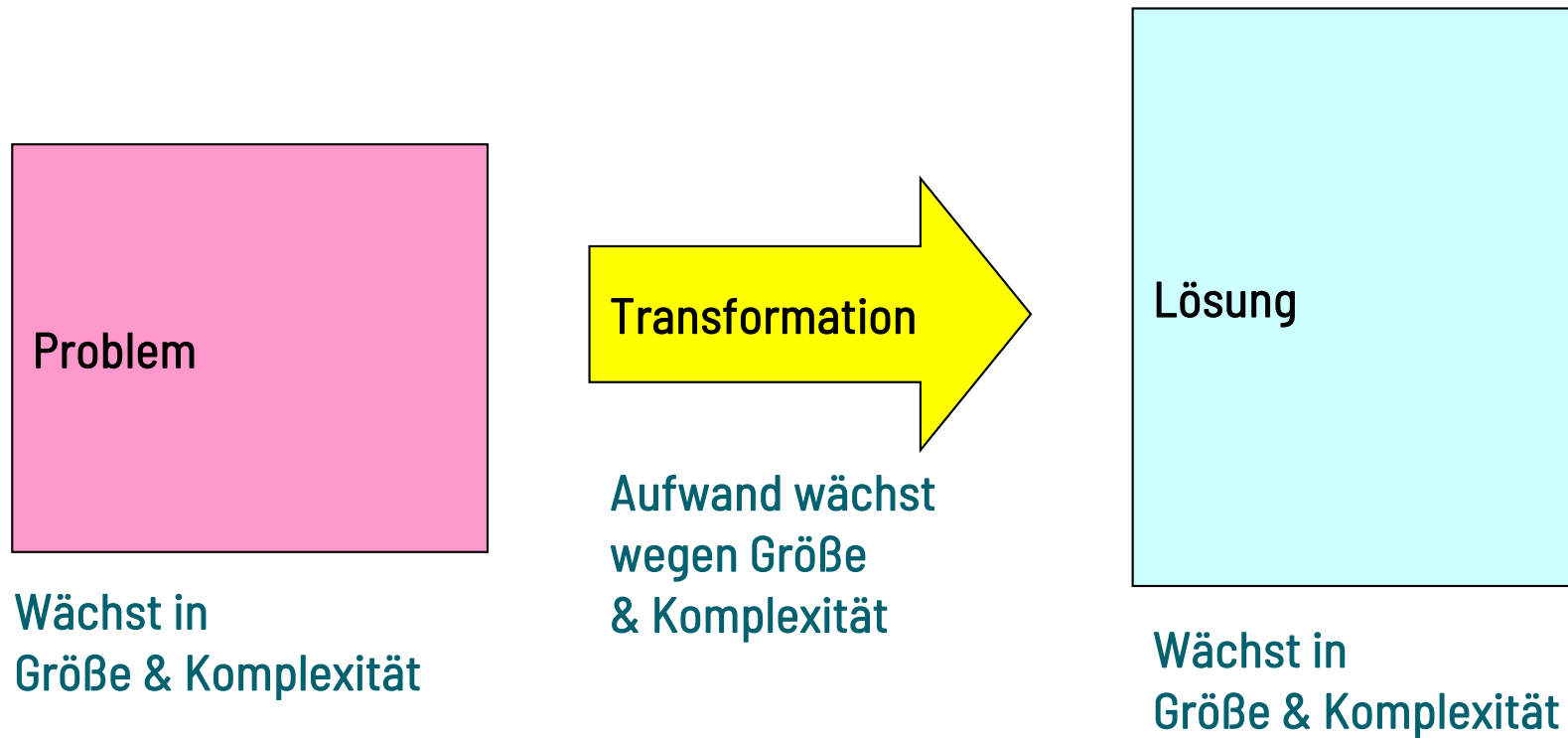
Nehmen Sie unser Beispiel: Was passiert, wenn Sie eine ganze Tabellenkalkulation mit 80 Berechnungsfunktionen und Schnittstellen zu anderer Office-Software erstellen müssen?

Komplexitätszunahme in der Tiefe

- Mehr **Details** in jeder Funktion
- Mehr Verknüpfungen und Verbindungen
 - **Teilen von Daten** zwischen Funktionen
 - **Steuerungsübergabe** zwischen Funktionen

Nehmen Sie unser Beispiel: Was passiert, wenn Sie auch den Durchschnitt von 60.000 Zahlen verarbeiten sollen und die Zahlen dazu auch noch richtig groß sein können?

Komplexität wächst an jeder Stelle



Software Engineering erklärt den
Schritt vom Programmieren zur
Entwicklung von Softwaresystemen.

Großen Softwaresystemen.

Dabei beschäftigen wir uns mit Methoden und Werkzeugen,
die normalerweise bei kleinen Programmen irrelevant sind.

Software ist überall

Wirtschaft und Alltag aller (nur halbwegs) entwickelten Staaten ist von Software höchst abhängig, und fast alles wird durch Software gesteuert:

- Im **Haus**: Küchengeräte, Heizung, Fernseher, Musikanlage, Spielen...
- **Unterwegs**: Verschiedenste Fahrzeugkomponenten, Verkehrsregelung, Verkehrsinformation, „Blitzer“
- In der **Hochschule**: LSF, Moodle, QM-Portal, Mensa-Plan, Website, ...
- **Kommunikation**: Telefonie allgemein, Handys, Messenger, ...
- **Finanzwirtschaft**: Bankgeschäfte, Börsenhandel, ...
- **Handel & Produktion**: Betriebliche Informationssysteme wie CRM, ERP, WaWi, Logistik, ...
- ...

Software ist besonders

- Komplex
- Immateriell
- Verformbar

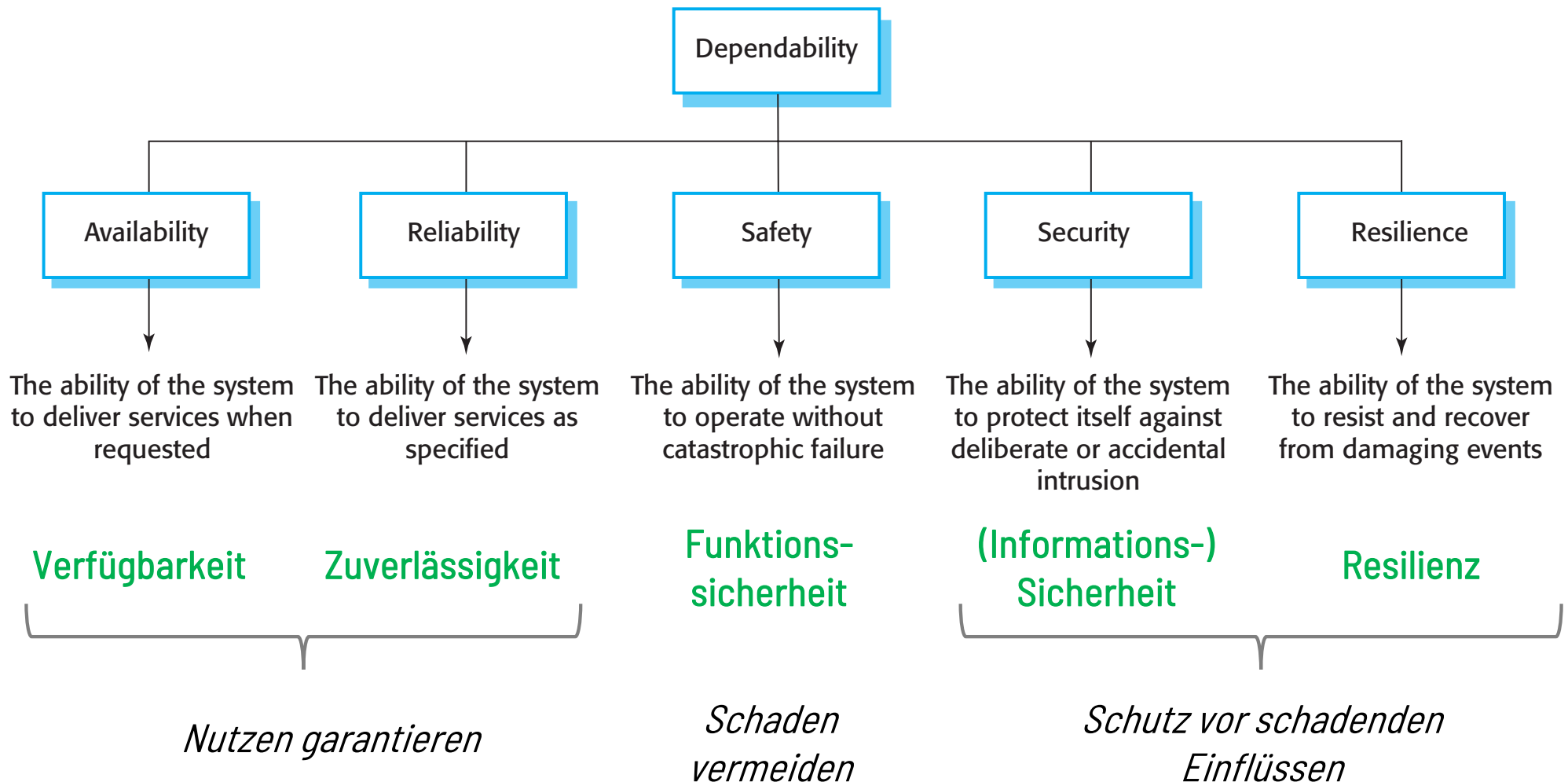
Software und „Sicherheit“

Beides hängt untrennbar zusammen

- Moderne Gesellschaften sind von **Software abhängig** und durchdrungen, Computersysteme international eng **verflochten**
- Damit entsteht ein **hoher Wert** – und folglich **Risiken**:
 - Systeme durch Fehler aus oder stehen nicht zur Verfügung
 - Daten gehen verloren oder werden absichtlich oder zufällig weltweit offengelegt oder manipuliert
 - Systeme sind Ziele organisierte Kriminalität
 - Systeme sind Ziele in internationalen Auseinandersetzungen
- **Wir müssen uns auf Computersysteme „verlassen“ können**, sie müssen „sicher“ sein – aber was heißt das genau?
- **Komplexität** erschwert es nicht nur, ein Ergebnis zu erzielen – **bedroht auch „Sicherheit“ und „Zuverlässigkeit“ des Ergebnisses**

Software und „Sicherheit“

Verlässliche Systeme, Sicherheit



Klein vs. Gross im Software Engineering 1

Klein	Groß
Programme von 1 bis ca. 300 Zeilen	Längere Programme
Für den Eigengebrauch	Für den Gebrauch durch Dritte
Vage Zielsetzung genügt	Genaue Zielbestimmung erforderlich
Ein Schritt vom Problem zur Lösung genügt: Lösung direkt programmiert	Mehrere Schritte erforderlich
Validierung am Endprodukt	Auf jeden Entwicklungsschritt muss ein Prüfschritt folgen
Eine Person entwickelt: Keine Koordination notwendig, keine Kommunikationsbedürfnisse	Mehrere Personen entwickeln gemeinsam: Koordination und Kommunikation notwendig

Klein vs. Gross im Software Engineering 2

Klein	Groß
Komplexität des Problems in der Regel klein	Komplexität des Problems größer bis sehr groß
Behalten der Übersicht nicht schwierig	Maßnahmen zur Strukturierung und Modularisierung erforderlich
Software besteht aus wenigen Komponenten	Software besteht aus vielen Komponenten; Maßnahmen zur Komponentenverwaltung erforderlich
In der Regel keine Dokumentation	Dokumentation zwingend
Keine Planung und Projektorganisation erforderlich	Planung und Projektorganisation zwingend erforderlich

Größe (Textzeilen) in der Literatur

J. Joyce: Ulysses.

x 25 = 1 Mio Zeilen

735 Seiten

Größe (Textzeilen) in der Literatur



Leo Tolstoj: Krieg und Frieden. 1536 Seiten.

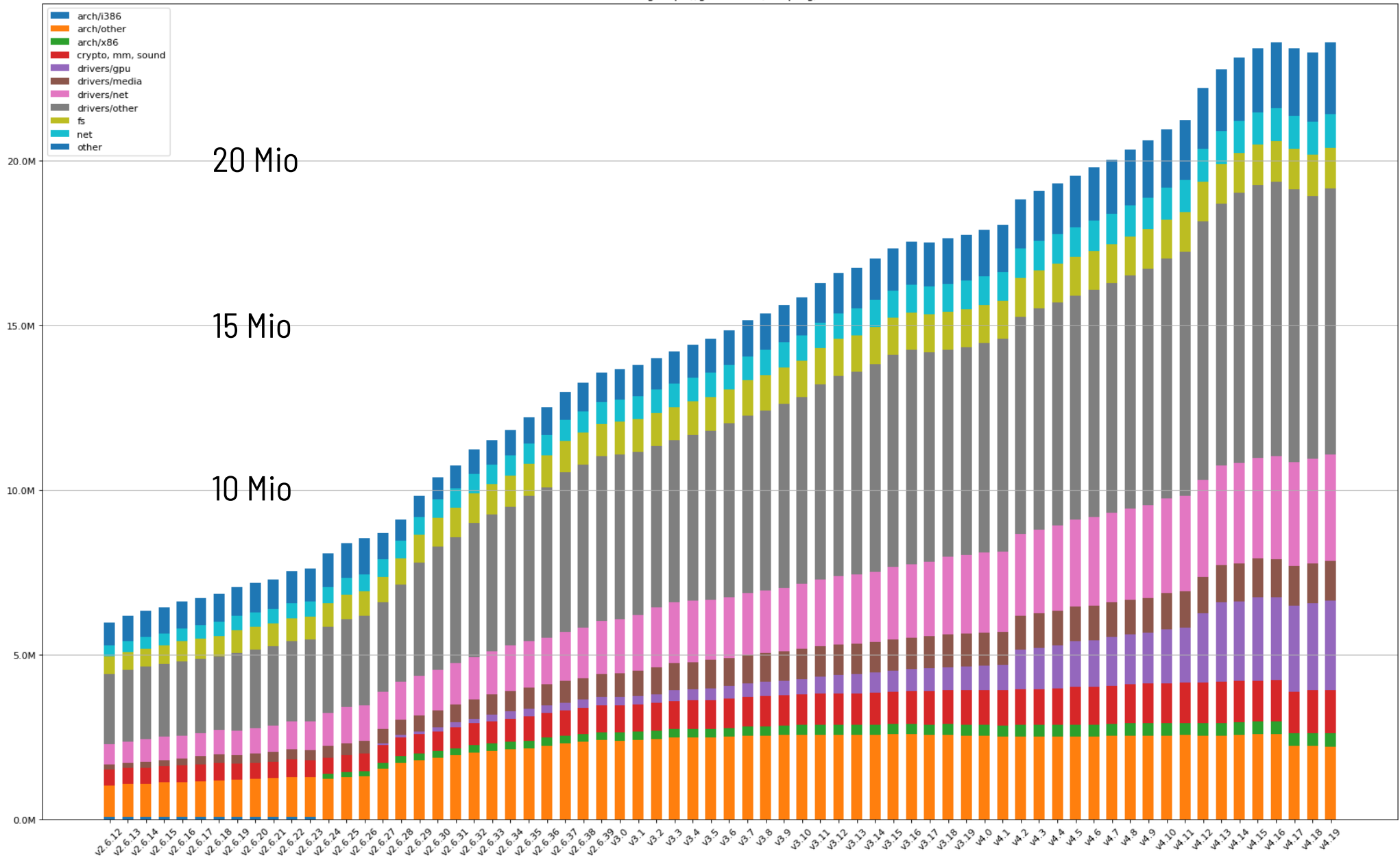
x 14 = 1 Mio Zeilen



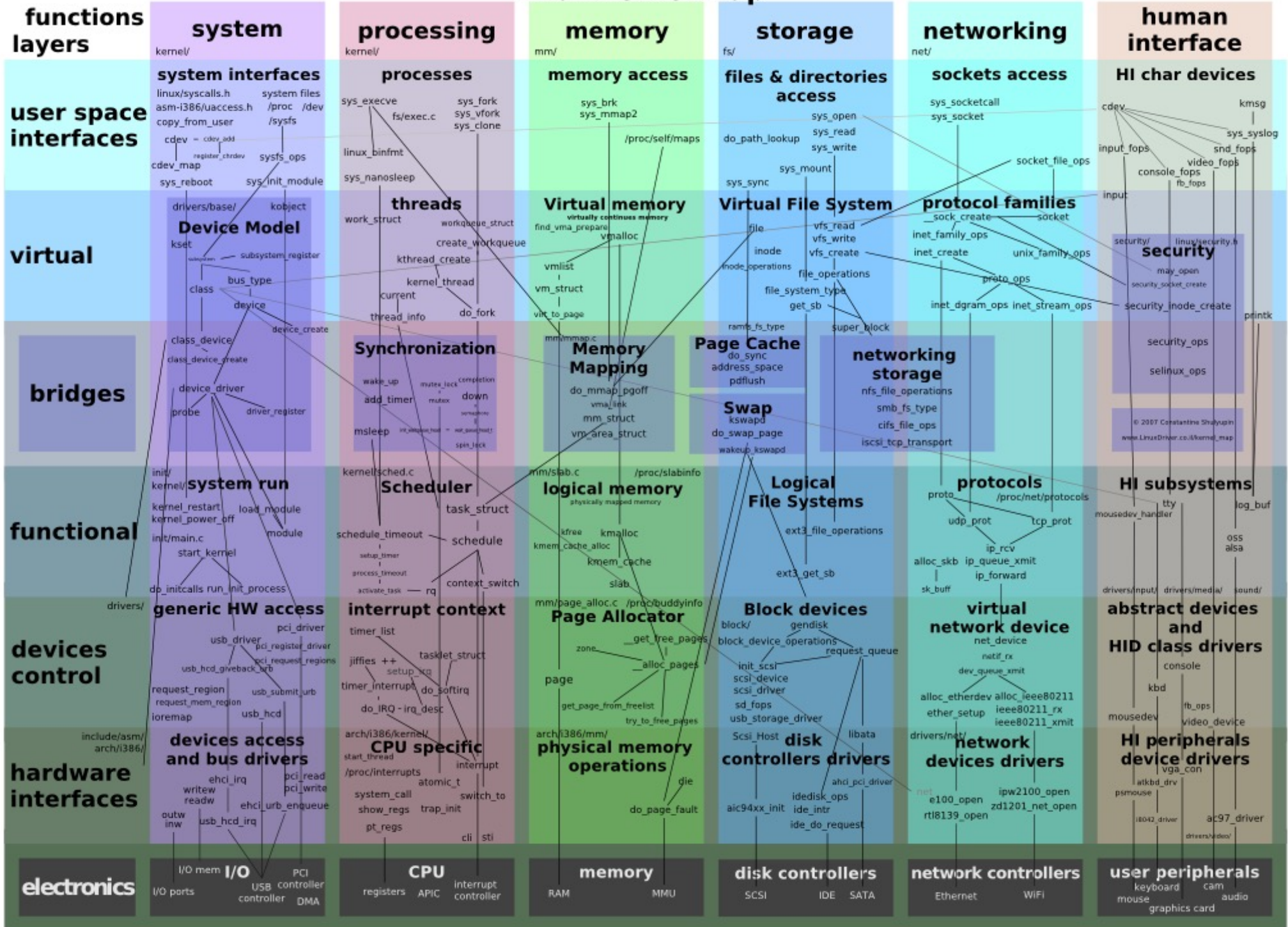
Entwicklung Linux 2.6.12 – 4.19

https://www.reddit.com/r/linux/comments/9uxwli/lines_of_code_in_the_linux_kernel/

Lines of code in the Linux kernel
Generated using <https://github.com/udoprog/kernelstats>



Linux kernel map



Statistiken zu Googles Quelltexten (2015!)

Software aller Google-Anwendungen und Dienste

@scale-Konferenz 2015, Rachel Potvin, Google

Google repository statistics

As of Jan 2015

Vorsicht: 1 billion (engl.) = 1 Milliarde (dt.)

Total number of files*	1 billion
Number of source files	9 million
Lines of code	2 billion
Depth of history	35 million commits
Size of content	86 terabytes
Commits per workday	45 thousand

*The total number of files includes source files copied into release branches, files that are deleted at the latest revision, configuration files, documentation, and supporting data files.

Statistiken zu Googles Quelltexten (2015!)

Some perspective

Linux kernel

- 15 million lines of code in 40 thousand files (total)

Google repository

- 15 million lines of code in 250 thousand files *changed per week, by humans*
- 2 billion lines of code, in 9 million source files (total)

Jede Woche
änderte sich
Googles
Quelltext in der
Größe des
kompletten
damaligen
Linux-Kernels

Grundbegriffe

Was ist Software eigentlich?

1. All or part of the programs, procedures, rules, and associated documentation of an information processing system.
2. Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.

ISO 24765 nach Laporte/April 2018

- Software entsteht durch ...
 - ...Erstellung
 - ...Konfiguration
 - ...Wiederverwendung

"Software is arguably the world's most important industry. The presence of software has made possible many new businesses and is responsible for increased efficiencies in most traditional businesses."

Grady Booch (Communications of the ACM, März 2001)

Grundbegriffe

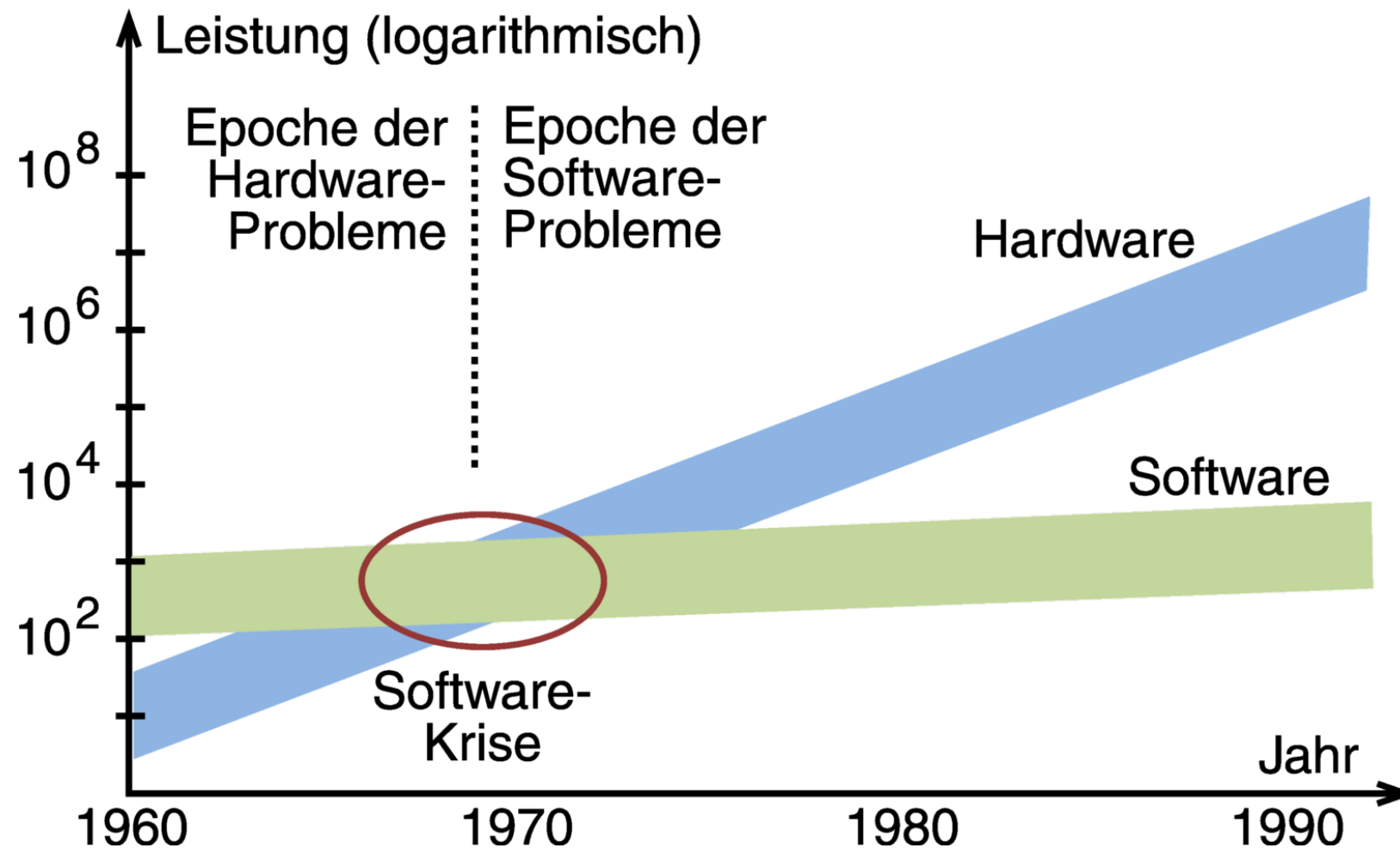
Software-System

Ein **Software-System** ist eine Menge von Software-Einheiten und ihren Beziehungen, wenn sie gemeinsam einem bestimmten Zweck dienen. Dieser Zweck ist im Allgemeinen komplex, er schließt neben der Bereitstellung eines ausführbaren Programms gewöhnlich die Organisation, Verwendung, Erhaltung und Weiterentwicklung des Programms ein.

Nach Ludewig / Lichter

- Unter einer „**Software-Einheit**“ verstehen wir Teile eines Systems, die nicht weiter zerlegbar sind oder nicht zerlegt werden sollen
- Ein **Software-Produkt** ist ein Produkt, das aus Software besteht

Fortschritte bei Hard- und Software Entwicklung von 1960 bis 1990



Quelle: Ludewig, Jochen; Lichter, Horst (2013): Software Engineering. Grundlagen, Menschen, Prozesse, Techniken. 3., korrigierte Aufl. Heidelberg: dpunkt.

Software Engineering: Der (historische) Weg zur Definition

"[...] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines."

– Nato-Konferenz, 1969



"[...] the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."

– IEEE, 1993



Software Engineering: Definition



„Zielorientierte Bereitstellung und systematische Verwendung von Prinzipien, Methoden und Werkzeugen für die arbeitsteilige, ingenieurmäßige Entwicklung und Anwendung von umfangreichen Softwaresystemen.“ – Balzert



"Software development as a series of resource-limited, goal-directed cooperative games of invention and communication." – Cockburn



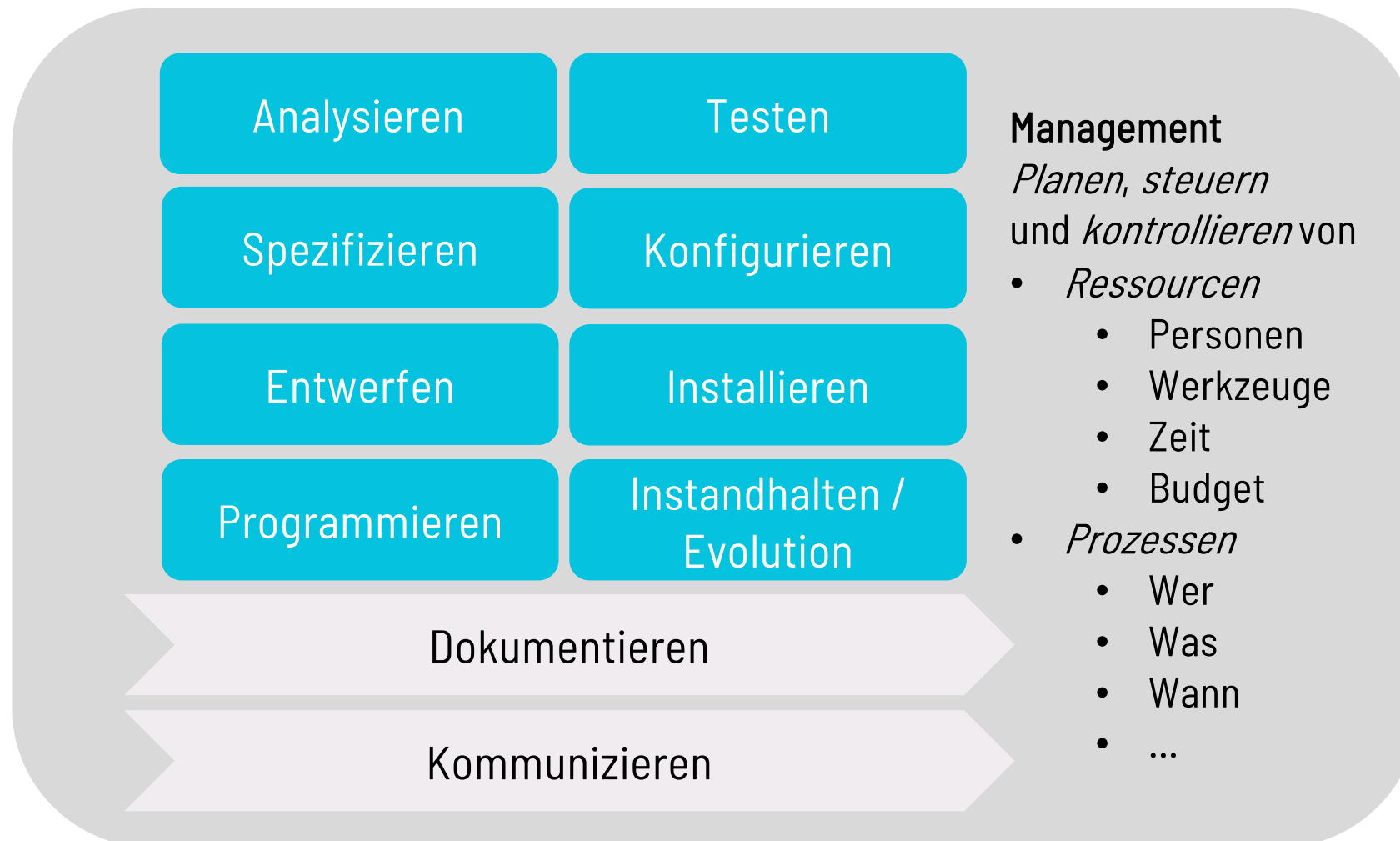
Software Engineering ist...

- **Professionelle Software-Entwicklung**
- Befaßt mit **allen Aspekten der Software-Produktion**
 - Von ersten Phasen der Systemspezifikation bis hin zur Weiterentwicklung des Systems nach Inbetriebnahme
 - Nicht nur technischer Entwicklungs-Prozess: Ebenso Prozess- und Projektsteuerung sowie Entwicklung geeigneter Methoden, Werkzeuge, Organisationsformen
- **Ingenieurmäßige Entwicklung:**
Verwendung geeigneter Theorien und Methoden zur Lösung von Problemen unter Berücksichtigung organisatorischer und finanzieller Einschränkungen

Professionelle, ingenieurmäßige Entwicklung: Kosteneffizienz!

- Kosten entstehen nicht nur mit der ersten Version einer Software: Die Wartung (Weiterentwicklung) kostet mehr als in Entwicklung der ersten Version
 - Für die meisten Systeme: Hauptsächliche Kosten fallen nach erster Inbetriebnahme an
 - Bei längerer Verwendungszeit: Wartungskosten betragen leicht ein vielfaches ursprünglicher Entwicklungskosten
- Auf längere Sicht: Verwendung von Methoden und Werkzeugen des Software Engineering günstiger als „naive Programmierung“
 - Start-Ups: Wie beginnt man? Wann stellt man um?

Aktivitäten im Software Engineering



Themen in dieser Vorlesung



Zusammenfassung

- Software durchdringt unser vernetztes Leben
- „reale“ Softwaresysteme sind relativ schnell relativ groß mit Anzahl von Quelltextzeilen in Hunderttausenden oder -zig Millionen, besonders bei
 - bei entsprechender Komplexität der gelösten Aufgabe,
 - hoher Funktionalität,
 - größerem Entwicklungsalter
- Entsprechend groß sind...
 - ...die Entwicklungsteams bzw. die Anzahl der Personen, die im Laufe der Zeit an dem System gearbeitet haben
 - ...die Herausforderungen an Vorgehensweise in der Entwicklung
- Dabei sind besonders Verlässlichkeit inklusive Sicherheit Herausforderungen

